

BỘ MÔN KHOA HỌC DỮ LIỆU

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

LAB 06 – ARTIFICIAL INTELLIGENCE



TS. TRẦN NGỌC VIỆT



HỌC KỲ 1 – NĂM HỌC 2024-2025



KHÓA K28 & K29

#Thực hành 01:

#LDA

+Phân tích phân biệt tuyến tính là gì?

- Trường hợp biết rằng trong khi xử lý tập dữ liệu nhiều chiều, ta phải áp dụng một số kỹ thuật giảm kích thước cho dữ liệu hiện có để có thể khám phá dữ liệu và sử dụng nó để lập mô hình một cách hiệu quả.
- Tìm hiểu về một kỹ thuật giảm kích thước và được sử dụng để ánh xạ dữ liệu có chiều cao sang chiều tương đối thấp hơn mà không mất nhiều dữ liệu.
- Phân tích phân biệt tuyến tính -LDA, gọi là Phân tích phân biệt chuẩn, là một kỹ thuật giảm kích thước, được sử dụng dạng bài toán có giám sát.
- Bài toán tạo điều kiện cho việc mô hình hóa sự khác biệt giữa các nhóm, phân tách hiệu quả hai hoặc nhiều lớp.
- LDA hoạt động bằng cách chiếu các đặc điểm từ không gian có chiều cao hơn vào không gian có chiều thấp hơn.
- LDA là thuật toán học có giám sát được thiết kế đặc biệt cho nhiệm vụ phân loại, nhằm xác định sự kết hợp tuyến tính của các tính năng giúp phân tách các lớp trong tập dữ liệu một cách tối ưu.
- Ví dụ:** dữ liệu có hai lớp và cần phân tách chúng một cách hiệu quả. Các lớp có thể có nhiều tính năng. Chỉ sử dụng một đặc điểm duy nhất để phân loại, có thể dẫn đến một số sự chồng chéo như trong hình bên dưới. Vì vậy, buộc phải tiếp tục tăng số lượng tính năng để phân loại phù hợp.

#Thực hành 01:

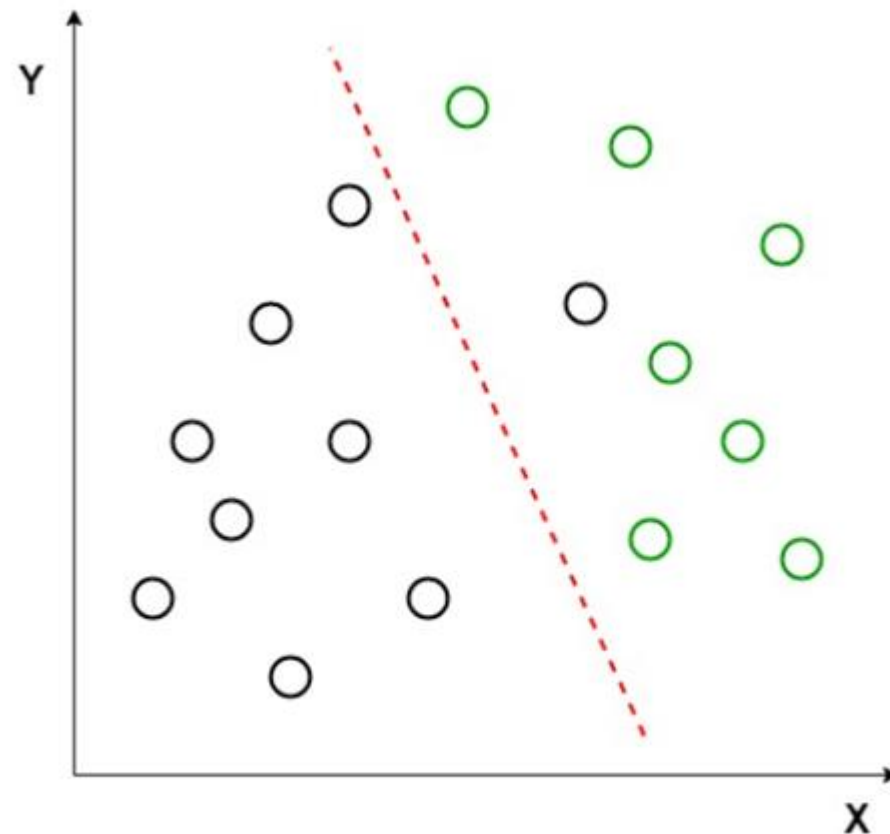
+Giả định của LDA

-LDA giả định rằng dữ liệu có phân phối Gaussian và ma trận hiệp phương sai của các lớp khác nhau là bằng nhau.

-LDA cũng giả định rằng dữ liệu có thể phân tách tuyến tính, là ranh giới quyết định tuyến tính có thể phân loại chính xác các lớp khác nhau.

-Giả sử có hai bộ điểm dữ liệu thuộc hai lớp khác nhau, cần phân loại. Hiển thị như biểu đồ 2D đã cho.

-Các điểm dữ liệu được vẽ trên mặt phẳng 2D, không có đường thẳng nào có thể phân tách hoàn toàn hai loại điểm dữ liệu. Do đó, trường hợp này phải dùng LDA được sử dụng để giảm biểu đồ 2D thành biểu đồ 1D, nhằm tối đa hóa khả năng phân tách giữa hai lớp.



Hình 1 – tập dữ liệu phân tách tuyến tính

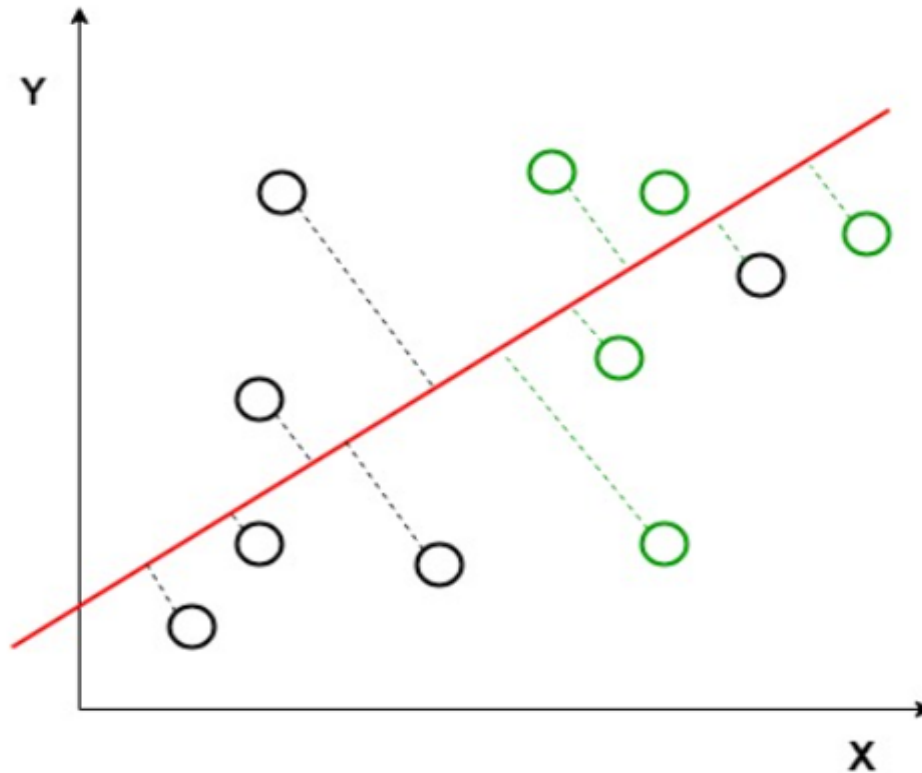
#Thực hành 01:

-Phân tích phân biệt tuyến tính sử dụng cả hai trục OX và OY để tạo trục mới và chiếu dữ liệu lên trục mới theo cách tối đa hóa sự tách biệt của hai lớp và giảm biểu đồ 2D thành biểu đồ 1D.

-Hai tiêu chí được LDA sử dụng để tạo trục mới:

*Tối đa hóa khoảng cách giữa sự tách biệt của hai lớp.

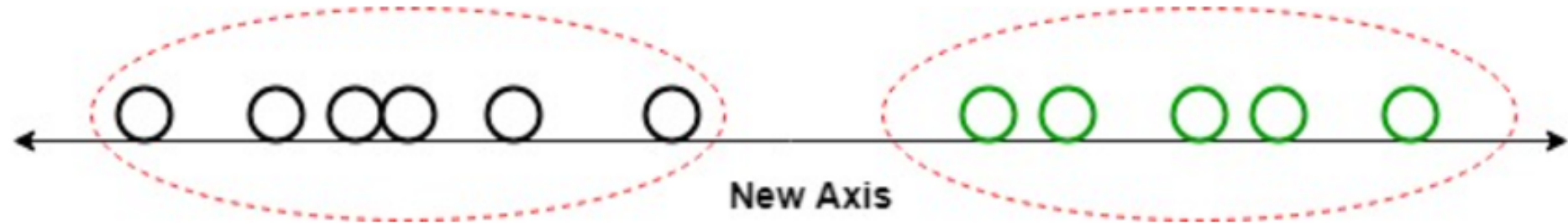
*Giảm thiểu sự khác biệt trong mỗi lớp.



Hình 2 – khoảng cách góc vuông giữa đường thẳng và điểm

#Thực hành 01:

- Biểu đồ trên, thấy rằng một trục mới được tạo và vẽ trong biểu đồ 2D sao cho nó tối đa hóa khoảng cách giữa giá trị trung bình của hai lớp và giảm thiểu sự biến đổi trong mỗi lớp.
- Trục mới được tạo, làm tăng sự tách biệt giữa các điểm dữ liệu của hai lớp.
- Tạo trục mới bằng cách sử dụng tiêu chí nêu trên, các điểm dữ liệu của các lớp được vẽ trên trục mới.



Hình 3 – khoảng cách giữa giá trị trung bình của 2 lớp

#Thực hành 01:

```
#LDA algorithm
#import thư viện hàm - Scikit Learn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

#dataset - Iris
iris = load_iris()
dataset = pd.DataFrame(columns=iris.feature_names,
                        data=iris.data)

dataset['target'] = iris.target

#Lớp dữ liệu & biến mục tiêu
X = dataset.iloc[:, 0:4].values
y = dataset.iloc[:, 4].values
```

#Thực hành 01:

```
#xử lý dữ liệu - huấn luyện và kiểm tra dữ liệu
sc = StandardScaler()
X = sc.fit_transform(X)
le = LabelEncoder()
y = le.fit_transform(y)
X_train, X_test, \
    y_train, y_test = train_test_split(X, y,
                                       test_size=0.2)

#LDA Phân tích tuyến tính
lda = LinearDiscriminantAnalysis(n_components=2)
X_train = lda.fit_transform(X_train, y_train)
X_test = lda.transform(X_test)
#đồ thị phân tán
plt.scatter(
    X_train[:, 0], X_train[:, 1],
    c=y_train,
    cmap='rainbow',
    alpha=0.7, edgecolors='b'
)
#sử dụng bộ phân loại rừng ngẫu nhiên;
classifier = RandomForestClassifier(max_depth=2,
                                   random_state=0)

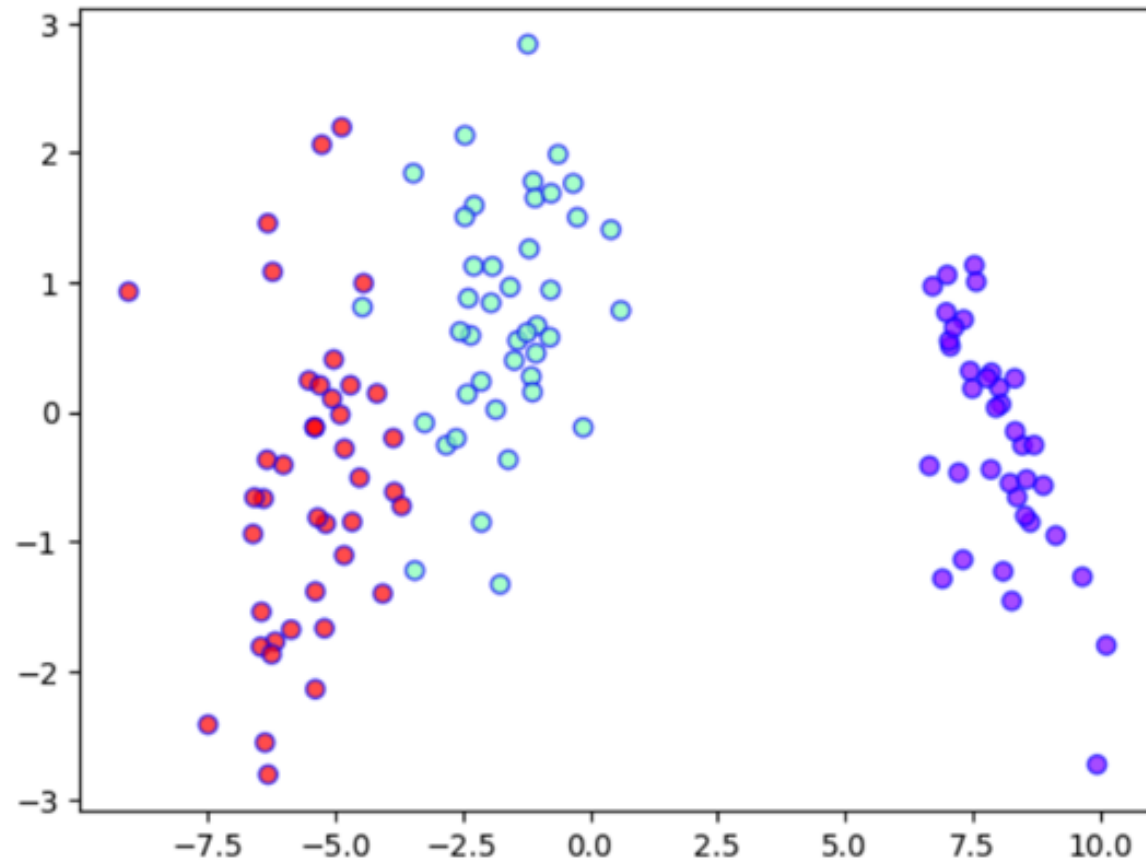
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
#ma trận nhầm lẫn & độ chính xác;
print('Accuracy : ' + str(accuracy_score(y_test, y_pred)))
conf_m = confusion_matrix(y_test, y_pred)
print(conf_m)
```

#Thực hành 01:

#out:

Accuracy : 1.0

```
[[13  0  0]  
 [ 0  8  0]  
 [ 0  0  9]]
```



#nhận xét: input, output?

#Thực hành 02:

```
#Gaussian Naive Bayes - ứng dụng Blockchain
import numpy as np
import pandas as pd
from sklearn.naive_bayes import GaussianNB

df = pd.read_csv('data_BC.csv')
print("Blocks of the Blockchain")
print (df.head())
#chuẩn bị tập huấn luyện
X = df.loc[:, 'DAY': 'BLOCKS']
Y = df.loc[:, 'DEMAND']
#chọn lớp - GaussianNB
clfG = GaussianNB()
#mô hình huấn luyện
clfG.fit(X,Y)
```

#Thực hành 02:

```
#dự đoán bằng mô hình
print("Blocks for the prediction of the A-F blockchain")
blocks=[ [12,2345,12],
          [13,2034,50],
          [25,7789,4],
          [27,6789,4]]

print(blocks)

prediction = clfG.predict(blocks)

for i in range(4):
    print("Block #",i+1," Gauss Naive Bayes Prediction:",prediction[i])
```

#Thực hành 02:

#out:

Blocks of the Blockchain

	DAY	STOCK	BLOCKS	DEMAND
0	10	1455	78	1
1	11	1666	67	1
2	12	1254	57	1
3	14	1563	45	1
4	15	1674	89	1

Blocks for the prediction of the A-F blockchain

[[12, 2345, 12], [13, 2034, 50], [25, 7789, 4], [27, 6789, 4]]

Block # 1 Gauss Naive Bayes Prediction: 1

Block # 2 Gauss Naive Bayes Prediction: 1

Block # 3 Gauss Naive Bayes Prediction: 2

Block # 4 Gauss Naive Bayes Prediction: 2

#nhận xét: input, output?

#Thực hành 03:

#K-means clustering

```
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
#load file V1.csv
df = pd.read_csv('V1.csv')
print (df.head())
#KNN-nhãn phân loại
X = df.loc[:, 'broke': 'shouted']
Y = df.loc[:, 'class']
#mô hình huấn luyện
knn = KNeighborsClassifier()
knn.fit(X,Y)
X_DL = [[9,0,9,0]]
prediction = knn.predict(X_DL)
print ("The prediction is:", str(prediction).strip('[]'))
#quy tắc từ vựng của tập dữ liệu
df = pd.read_csv('V1.csv')
#biểu đồ- mối quan hệ của từng đặc điểm với từng lớp
figure, (sub1, sub2, sub3, sub4)=plt.subplots(4, sharex=True, sharey=True)
plt.suptitle('k-nearest neighbors')
```

#Thực hành 03:

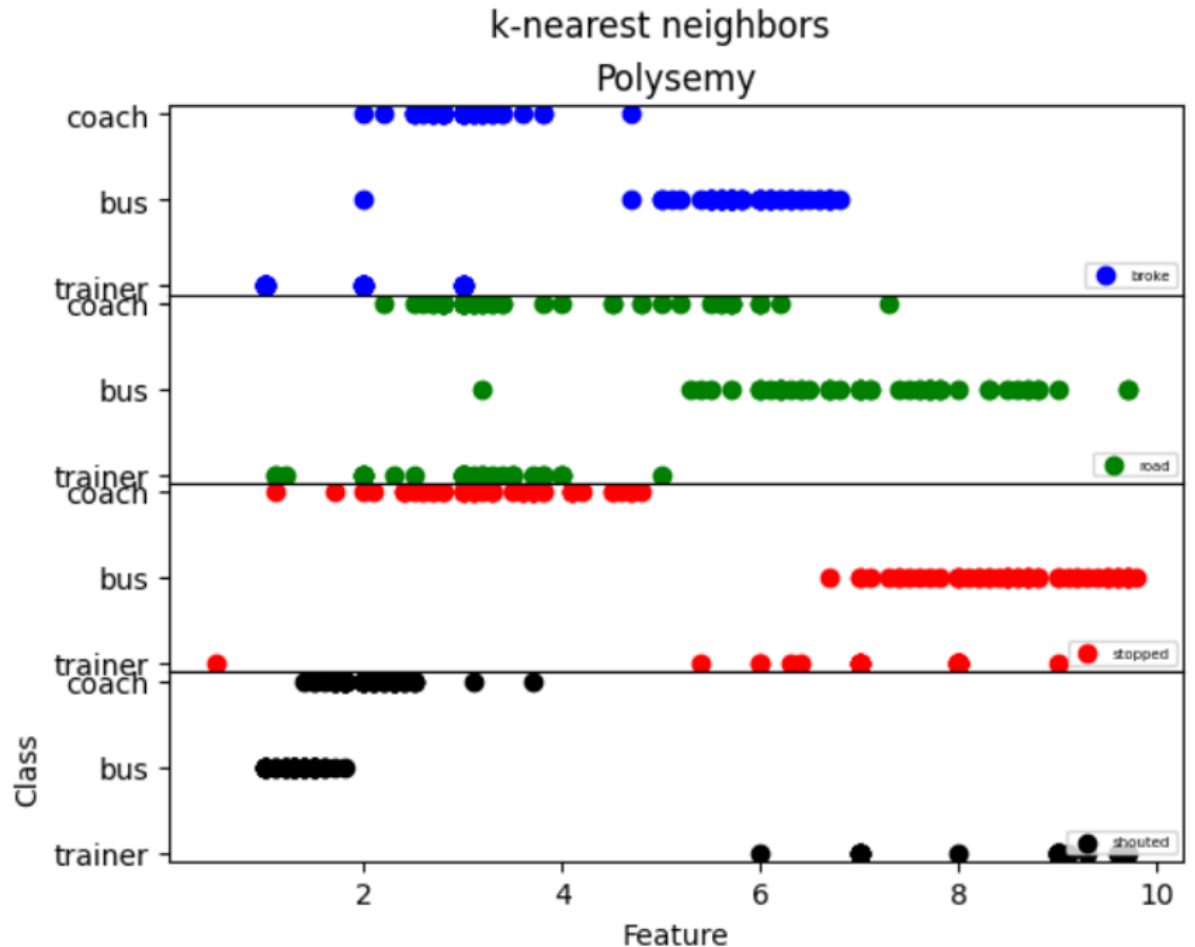
```
plt.xlabel('Feature')
plt.ylabel('Class')
X = df.loc[:, 'broke']
Y = df.loc[:, 'class']
sub1.scatter(X, Y, color='blue', label='broke')
sub1.legend(loc=4, prop={'size': 5})
sub1.set_title('Polysemy')
X = df.loc[:, 'road']
Y = df.loc[:, 'class']
sub2.scatter(X, Y, color='green', label='road')
sub2.legend(loc=4, prop={'size': 5})
X = df.loc[:, 'stopped']
Y = df.loc[:, 'class']
sub3.scatter(X, Y, color='red', label='stopped')
sub3.legend(loc=4, prop={'size': 5})
X = df.loc[:, 'shouted']
Y = df.loc[:, 'class']
sub4.scatter(X, Y, color='black', label='shouted')
sub4.legend(loc=4, prop={'size': 5})
figure.subplots_adjust(hspace=0)
plt.show()
```

#Thực hành 03:

#out:

	broke	road	stopped	shouted	class
0	1.0	3.5	6.4	9.0	trainer
1	1.0	3.0	5.4	9.0	trainer
2	1.0	3.2	6.3	9.0	trainer
3	1.0	3.1	0.5	9.0	trainer
4	1.0	2.0	8.0	9.0	trainer

The prediction is: 'bus'



#nhận xét: input, output?

#Thực hành 04:

```
#K-means clustering with Mini-Batch
from sklearn.cluster import KMeans
import pandas as pd
from matplotlib import pyplot as plt
from random import randint
import numpy as np
#file data.csv
dataset = pd.read_csv('data.csv')
print (dataset.head())
print(dataset)
n=1000
dataset1=np.zeros(shape=(n,2))
li=0
for i in range (n):
    j=randint(0,4999)
    dataset1[li][0]=dataset.iloc[j,0]
    dataset1[li][1]=dataset.iloc[j,1]
    li+=1
```

#Thực hành 04:

```
#siêu tham số
k = 6
kmeans = KMeans(n_clusters=k)
#thuật toán k-means
kmeans = kmeans.fit(dataset1)
gcenters = kmeans.cluster_centers_    #các hình của trung tâm
print("The geometric centers or centroids:")
print(gcenters)
#xác định nhãn
labels = kmeans.labels_
colors = ['blue', 'red', 'green', 'black', 'yellow', 'brown', 'orange']
#kết quả: điểm dữ liệu và cụm
y = 0
for x in labels:
    plt.scatter(dataset1[y,0], dataset1[y,1], color=colors[x])
    y+=1
for x in range(k):
    lines = plt.plot(gcenters[x,0], gcenters[x,1], 'kx')
```


#Thực hành 04:

```
title = ('No of clusters (k) = {}'.format(k))
plt.title(title)
plt.xlabel('Distance')
plt.ylabel('Location')
plt.show()

#kiểm tra tập dữ liệu và dự đoán
x_test = [[40.0, 67], [20.0, 61], [90.0, 90],
           [50.0, 54], [20.0, 80], [90.0, 60]]
prediction = kmeans.predict(x_test)
print("The predictions:")
print(prediction)
```

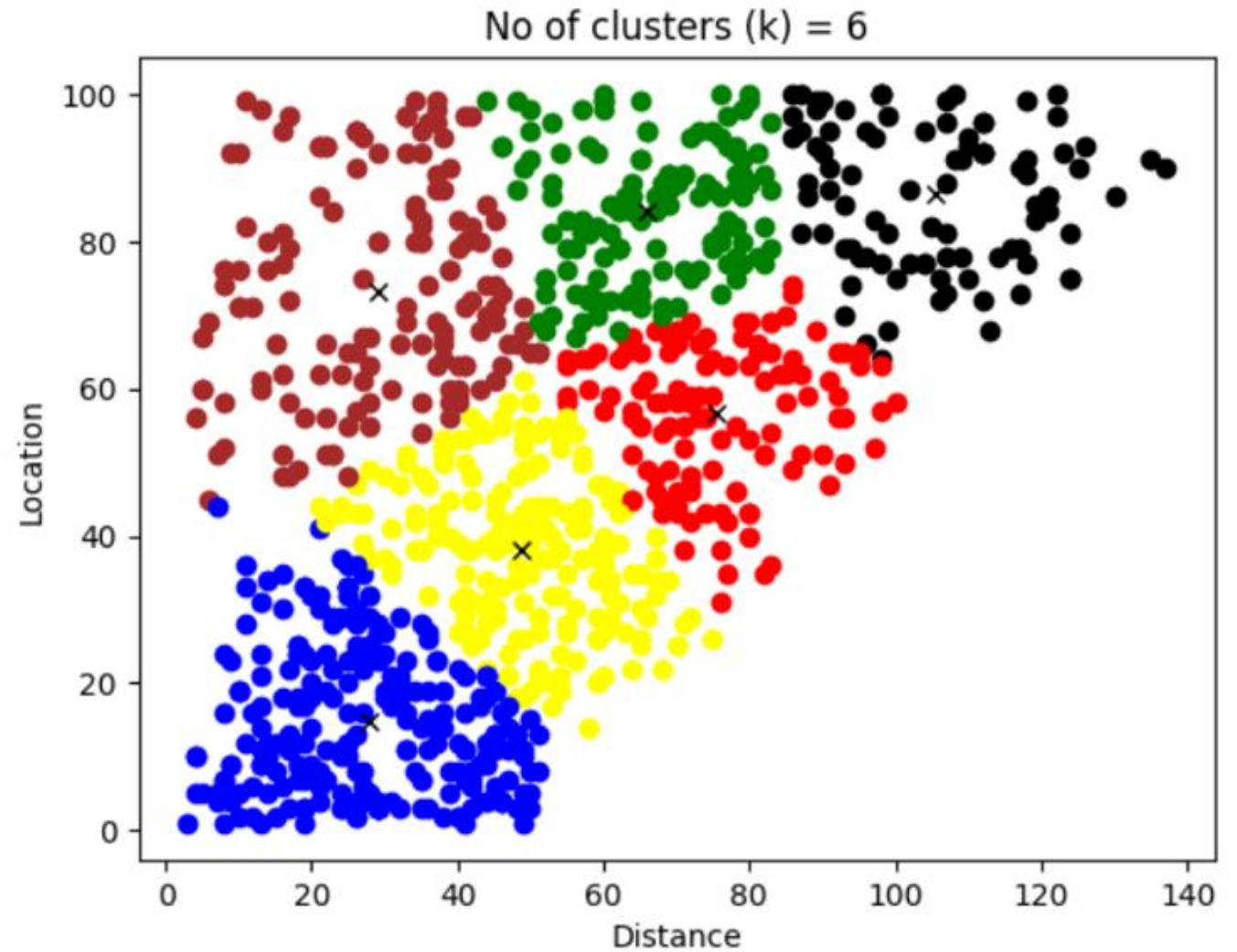
#Thực hành 04:

#out:

	Distance	location
0	80	53
1	18	8
2	55	38
3	74	74
4	17	4

	Distance	location
0	80	53
1	18	8
2	55	38
3	74	74
4	17	4
...	...	...
4994	26	28
4995	50	91
4996	19	32
4997	73	95
4998	36	64

[4999 rows x 2 columns]
The geometric centers or centroids:
[[33.15882353 55.37647059]
[71.25139665 60.43575419]
[19.9483871 15.89032258]
[47.265625 87.546875]
[96.6993007 84.20979021]
[49.18222222 23.97333333]]



The predictions:
[5 5 3 4 5 1]

#nhận xét: input, output?



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

